

Project Proposal

Explainable Machine Learning for Network Intrusion Detection with LLM-Augmented Incident Triage

September 14, 2026

1. Concept Summary

You will build a supervised ML pipeline that classifies network traffic into benign vs. specific attack categories (DoS, Probe, R2L, U2R) using a standard intrusion-detection benchmark, then layer explainability (SHAP) on top so every flagged prediction comes with the features that drove it.

The final layer feeds that structured evidence into a Large Language Model (LLM), which converts a raw model output into a plain-English incident summary, a severity rating, and a recommended response action—turning a black-box classifier into something that behaves like a junior SOC analyst’s first-pass triage.

2. Core Technical Stack & Implementation Details

2.1. Stack

- **Language:** Python 3.x, developed in Jupyter/Colab
- **Data handling:** pandas, NumPy
- **Classical ML:** `scikit-learn` (Logistic Regression, Random Forest, metrics/train-test splitting)
- **Neural net:** TensorFlow/Keras for a small feedforward network—the same framework the DeepLearning.AI Specialization uses, so this is a direct, honest extension of coursework you’ve already done.
- **Explainability:** SHAP
- **LLM layer:** Anthropic API (Claude)—chosen deliberately because it’s the exact API named in your Rank 8 project (Al Maruf), so you’ll already have hands-on experience with the tool his lab uses; OpenAI’s API is a drop-in alternative if you prefer.
- **Optional demo:** Streamlit (cut first if week 4 gets tight—a clean notebook with sample outputs is enough).

2.2. Dataset

NSL-KDD as your primary dataset. It’s well-documented, right-sized for a solo 4-week build, and—not a coincidence—it’s one of three datasets named explicitly in your Rank 2 project’s (Aighobahi) own description, alongside CICIDS-2018 and TON-IoT, either of which works as a drop-in swap if you want more modern traffic patterns.

Note for later: this same pipeline (tabular features → classifier → SHAP → LLM explanation) is directly portable to a feature-based malware dataset like EMBER with just a dataset swap—you don’t need to build that now, but it’s worth mentioning in your applications as evidence the architecture generalizes.

3. 4-Week Roadmap

Week 1 — Data & Baseline Models

Load and clean NSL-KDD, encode categorical fields, normalize numeric ones, and do quick EDA (class balance, feature correlations). Train three baselines for multi-class attack classification: Logistic Regression, Random Forest, and a small Keras feedforward net. Evaluate with precision/recall/F1 per class and a confusion matrix; pick the strongest, most balanced model to carry forward.

Week 2 — Explainability Layer

Add SHAP on top of your chosen model and extract the top contributing features per prediction. Sanity-check that the model’s “reasoning” lines up with known signatures (e.g., DoS traffic driving on connection-count features). Package this as one reusable function: `explain(sample)` → `{prediction, confidence, top_features}`.

Week 3 — LLM-Augmented Triage

Design a prompt template that takes Week 2’s output plus a short reference description of each NSL-KDD attack category (a simple retrieval step) and asks the LLM to output a plain-English explanation, a severity rating, and a recommended action. Wire this to the Anthropic API in Python, batch-test on 30–50 flagged samples, and manually review for hallucination before iterating on the prompt.

Week 4 — Comparative Experiment & Packaging

Run a small side experiment: give the LLM only the raw feature description (no trained classifier) and have it classify a held-out sample set zero/few-shot; compare its accuracy against your trained model. Build a lightweight demo (or just a clean notebook) showing `sample` → `verdict` → `SHAP explanation` → `LLM triage note`. Write a short README covering methodology, results, and limitations, and push it to GitHub—this becomes a linkable artifact in your applications.